

CS4423 - Networks

Prof. Götz Pfeiffer

School of Mathematics, Statistics and Applied Mathematics

NUI Galway

2. Centrality Measures

Lecture 9: Closeness and Betweenness Centrality

We study two more centrality measures

- **closeness** centrality and
- **betweenness** centrality

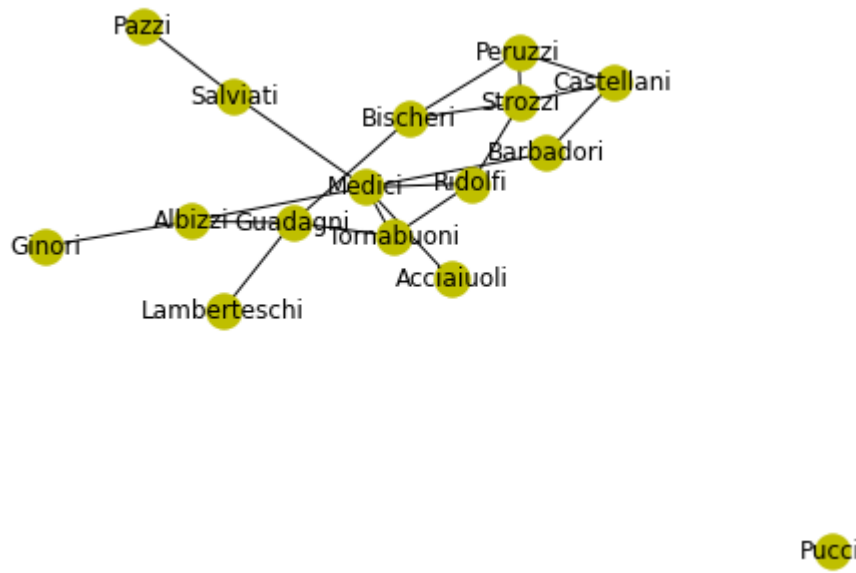
and compare these to the centralities studied earlier in the example of the marital ties graph of the Florentine families.

Import the packages (including a `Queue` for BFS) and set standard drawing options:

```
In [1]: import networkx as nx
import pandas as pd
from queue import Queue
opts = { "with_labels": True, "node_color": 'y' }
```

Next, recover the graph `G` of marital ties between Florentine families, together with the node attributes we have already determined.

```
In [2]: G = nx.read_yaml("data/florentine.yaml")
        nx.draw(G, **opts)
```



```
In [3]: G.nodes['Medici']
```

```
Out[3]: {'$C_i^D$': 0.4,
         '$c_i^E$': 0.4303154258349922,
         'degree': 6,
         'priorates': 53,
         'wealth': 103}
```

```
In [4]: G.number_of_nodes()
```

```
Out[4]: 16
```

Closeness Centrality

A node x in a network can be regarded as being central, if it is **close** to (many) other nodes, as it can then quickly interact with them. A simple way to measure closeness in this sense is based on the sum of all the distances to the other nodes, as follows.

Definition (Closeness Centrality). In a simple, **connected** graph G , the **closeness centrality** c_i^C of node i is defined as

$$c_i^C = \left(\sum_j d_{ij} \right)^{-1}.$$

The **normalized closeness centrality** of node i , defined as

$$C_i^C = (n-1)c_i^C$$

takes values in the interval $[0, 1]$.

How to compute c_i^C ?

BFS again.

- The following python function implements BFS for shortest distance from a previous lecture.
- It takes a graph $G = (X, E)$ and a vertex $x \in X$ as its arguments.
- It returns a **dictionary**, which assigns to each node its distance to x .

```
In [5]: def distances(G, x):

    # 1. init: set up the dictionary and a queue
    dists = { y: None for y in G }
    Q = Queue()
    dists[x] = 0
    Q.put(x)

    # 2. loop
    while not Q.empty():
        y = Q.get()
        for z in G.neighbors(y):
            if dists[z] is None:
                dists[z] = dists[y] + 1
                Q.put(z)

    # 3. stop here
    return dists
```

- For example, the distance from node 'Medici' to all nodes in G :

```
In [6]: d = distances(G, 'Medici')
print(d)
```

```
{'Acciaiuoli': 1, 'Albizzi': 1, 'Barbadori': 1, 'Bischeri': 3, 'Castellani': 2, 'Ginori': 2, 'Guadagni': 2, 'Lamberteschi': 3, 'Medici': 0, 'Pazzi': 2, 'Peruzzi': 3, 'Pucci': None, 'Ridolfi': 1, 'Salviati': 1, 'Strozzi': 2, 'Tornabuoni': 1}
```

- The isolated node 'Pucci' :

```
In [7]: d = distances(G, 'Pucci')
print(d)
```

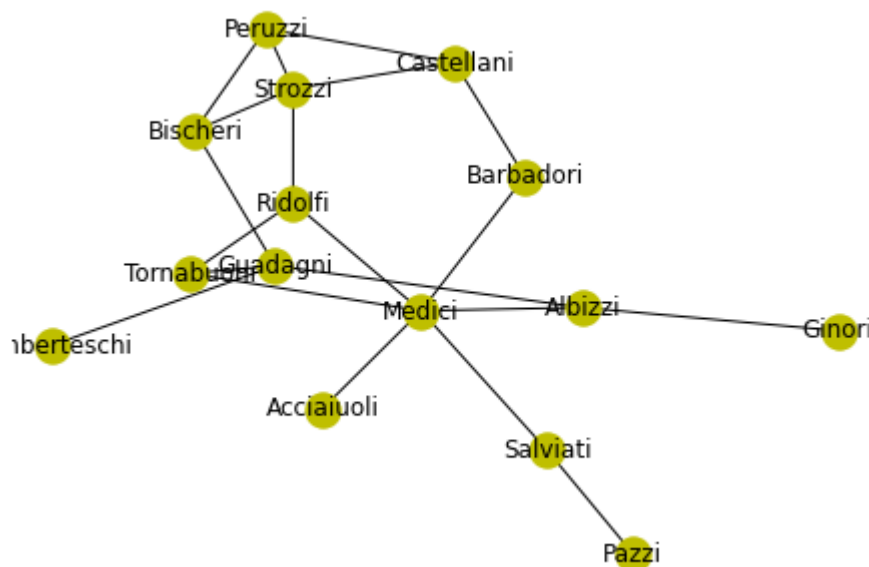
```
{'Acciaiuoli': None, 'Albizzi': None, 'Barbadori': None, 'Bischeri': None, 'Castellani': None, 'Ginori': None, 'Guadagni': None, 'Lamberteschi': None, 'Medici': None, 'Pazzi': None, 'Peruzzi': None, 'Pucci': 0, 'Ridolfi': None, 'Salviati': None, 'Strozzi': None, 'Tornabuoni': None}
```

- If the sum of the distances is 0 (why?), computing the closeness will most likely cause a **division-by-zero error**.
- So let's remove the isolated node from `G` and update `n`, the number of nodes.

```
In [8]: G.remove_node('Pucci')
n = G.number_of_nodes()
n
```

```
Out[8]: 15
```

```
In [9]: nx.draw(G, **opts)
```



- Use `distances` to compute the normalized closeness centrality according to the above definition.

```
In [10]: closeness = { x : (n-1)/sum(distances(G, x).values()) for x in G }
closeness
```

```
Out[10]: {'Acciaiuoli': 0.3684210526315789,
'Albizzi': 0.4827586206896552,
'Barbadori': 0.4375,
'Bischeri': 0.4,
'Castellani': 0.3888888888888889,
'Ginori': 0.3333333333333333,
'Guadagni': 0.4666666666666667,
'Lamberteschi': 0.32558139534883723,
'Medici': 0.56,
'Pazzi': 0.2857142857142857,
'Peruzzi': 0.3684210526315789,
'Ridolfi': 0.5,
'Salviati': 0.3888888888888889,
'Strozzi': 0.4375,
'Tornabuoni': 0.4827586206896552}
```

- Compare the results to the `networkx` version of closeness:

```
In [11]: nx.closeness centrality(G)
```

```
Out[11]: {'Acciaiuoli': 0.3684210526315789,  
          'Albizzi': 0.4827586206896552,  
          'Barbadori': 0.4375,  
          'Bischeri': 0.4,  
          'Castellani': 0.3888888888888889,  
          'Ginori': 0.3333333333333333,  
          'Guadagni': 0.4666666666666667,  
          'Lamberteschi': 0.32558139534883723,  
          'Medici': 0.56,  
          'Pazzi': 0.2857142857142857,  
          'Peruzzi': 0.3684210526315789,  
          'Ridolfi': 0.5,  
          'Salviati': 0.3888888888888889,  
          'Strozzi': 0.4375,  
          'Tornabuoni': 0.4827586206896552}
```

```
In [12]: nx.closeness centrality(G) == closeness
```

```
Out[12]: True
```

- Let's add those measurements to the table.

```
In [13]: nx.set_node_attributes(G, closeness, '$C_i^C$')
```

```
In [14]: pd.DataFrame.from_dict(
          dict(G.nodes(data=True)),
          orient='index'
        ).sort_values('degree', ascending=False)
```

```
Out[14]:
```

	C_i^D	c_i^E	degree	priorates	wealth	C_i^C
Medici	0.400000	0.430315	6	53	103	0.560000
Guadagni	0.266667	0.289117	4	21	8	0.466667
Strozzi	0.266667	0.355973	4	74	146	0.437500
Albizzi	0.200000	0.243961	3	65	36	0.482759
Bischeri	0.200000	0.282794	3	12	44	0.400000
Castellani	0.200000	0.259020	3	22	20	0.388889
Peruzzi	0.200000	0.275722	3	42	49	0.368421
Ridolfi	0.200000	0.341554	3	38	27	0.500000
Tornabuoni	0.200000	0.325847	3	n/a	48	0.482759
Barbadori	0.133333	0.211706	2	n/a	55	0.437500
Salviati	0.133333	0.145921	2	35	10	0.388889
Acciaiuoli	0.066667	0.132157	1	53	10	0.368421
Ginori	0.066667	0.074925	1	n/a	32	0.333333
Lamberteschi	0.066667	0.088793	1	0	42	0.325581
Pazzi	0.066667	0.044815	1	n/a	48	0.285714

Betweenness Centrality

When interactions between non-adjacent agents in a network depend on middle men (sitting on shortest paths between these agents), **power comes to those in the middle**. Betweenness centrality measures centrality in terms of the number of shortest paths a node lies on.

Definition (Betweenness Centrality). In a simple, connected graph G , the **betweenness centrality** c_i^B of node i is defined as

$$c_i^B = \sum_{j \neq i} \sum_{k \neq i} \frac{n_{jk}(i)}{n_{jk}},$$

where n_{jk} denotes the **number** of shortest paths from node j to node k , and where $n_{jk}(i)$ denotes the number of those shortest paths **passing through** node i .

The **normalized betweenness centrality** of node i , defined as

$$C_i^B = \frac{c_i^B}{(n-1)(n-2)}$$

takes values in the interval $[0, 1]$.

- Note that $(n-1)(n-2)$ is the largest number of shortest paths between pairs of distinct nodes that a given node could possibly sit on.
- How to compute C_i^B ?

BFS once more. This time as a python function, which returns a **dictionary** that contains, for each node y , a list of **immediate predecessors** of y in a shortest path from x to y . We have already seen that this another piece of information that BFS can determine on the fly: when constructing a **spanning tree** while traversing a graph, we need to remember **one** immediate predecessor for each newly discovered node. Here we determine and remember **all** immediate predecessors, requiring little if no extra work.

From this list of predecessors, one can then recursively reconstruct **all shortest paths** from x to y . We still need to keep track of the shortest path lengths in order to decide which neighbor x actually is a predecessor of y : .

```
In [15]: def predecessors(G, x):

    # 1. init: set up the two dictionaries and queue
    dists = { y: None for y in G }
    preds = { y: [] for y in G }
    Q = Queue()
    dists[x] = 0
    Q.put(x)

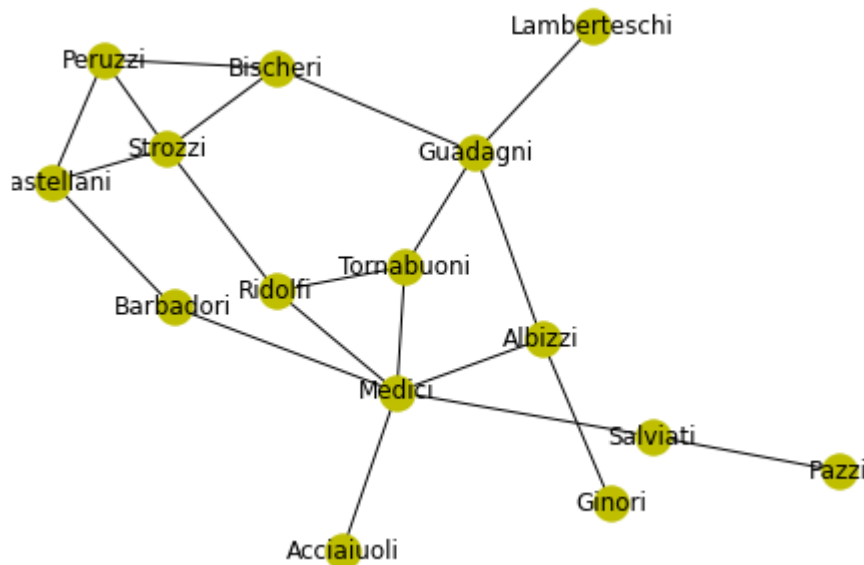
    # 2. loop
    while not Q.empty():
        y = Q.get()
        for z in G.neighbors(y):
            if dists[z] is None:
                dists[z] = dists[y] + 1
                preds[z].append(y)
                Q.put(z)
            elif dists[z] > dists[y]:
                preds[z].append(y)

    # 3. stop here
    return preds
```

```
In [16]: p = predecessors(G, 'Medici')
p
```

```
Out[16]: {'Acciaiuoli': ['Medici'],
'Albizzi': ['Medici'],
'Barbadori': ['Medici'],
'Bischeri': ['Guadagni', 'Strozzi'],
'Castellani': ['Barbadori'],
'Ginori': ['Albizzi'],
'Guadagni': ['Albizzi', 'Tornabuoni'],
'Lamberteschi': ['Guadagni'],
'Medici': [],
'Pazzi': ['Salviati'],
'Peruzzi': ['Castellani', 'Strozzi'],
'Ridolfi': ['Medici'],
'Salviati': ['Medici'],
'Strozzi': ['Ridolfi'],
'Tornabuoni': ['Medici']}
```

```
In [17]: nx.draw(G, **opts)
```



Using the **predecessor lists** with respect to x , the **shortest paths** from x to y can be enumerated recursively:

- if $y = x$: the shortest path from x to itself is the empty path starting and ending at x .
- else, if $y \neq x$ then each shortest path from x to y travels through exactly one of y 's predecessors ... and ends in y .

So, in formulas, $S_x(x) = \{(x)\}$ and

$$S_x(y) = \{p + (y) : p \in S_x(z), z \in \text{pre}_x(y)\}$$


```
In [18]: def shortest_paths(x, y, pre):
        if x == y:
            return [[x]]
        paths = []
        for z in pre[y]:
            for path in shortest_paths(x, z, pre):
                paths.append(path + [y])
        return paths
```

```
In [19]: def spaths(x, y, pre):
        if x == y:
            return [[x]]
        return [ p + [y] for p in spaths(x, z, pre) for z in pre[y] ]
```

```
In [20]: shortest_paths('Medici', 'Bischeri', p)
```

```
Out[20]: [['Medici', 'Albizzi', 'Guadagni', 'Bischeri'],
          ['Medici', 'Tornabuoni', 'Guadagni', 'Bischeri'],
          ['Medici', 'Ridolfi', 'Strozzi', 'Bischeri']]
```

- Now compute betweenness:

```
In [21]: betweenness = { x : 0.0 for x in G }
```

```
In [22]: for x in G:
        pre = predecessors(G, x)
        for y in G:
            paths = shortest_paths(x, y, pre)
            njk = len(paths)*(n-1)*(n-2) # normalize
            for p in paths:
                for z in p[1:-1]: # exclude endpoints
                    betweenness[z] += 1/njk
```

```
In [23]: betweenness
```

```
Out[23]: {'Acciaiuoli': 0.0,
          'Albizzi': 0.2124542124542123,
          'Barbadori': 0.09340659340659344,
          'Bischeri': 0.10439560439560439,
          'Castellani': 0.05494505494505494,
          'Ginori': 0.0,
          'Guadagni': 0.25457875457875445,
          'Lamberteschi': 0.0,
          'Medici': 0.521978021978021,
          'Pazzi': 0.0,
          'Peruzzi': 0.02197802197802198,
          'Ridolfi': 0.11355311355311352,
          'Salviati': 0.14285714285714285,
          'Strozzi': 0.10256410256410255,
          'Tornabuoni': 0.09157509157509156}
```

- Compare the results to the `networkx` version of betweenness:

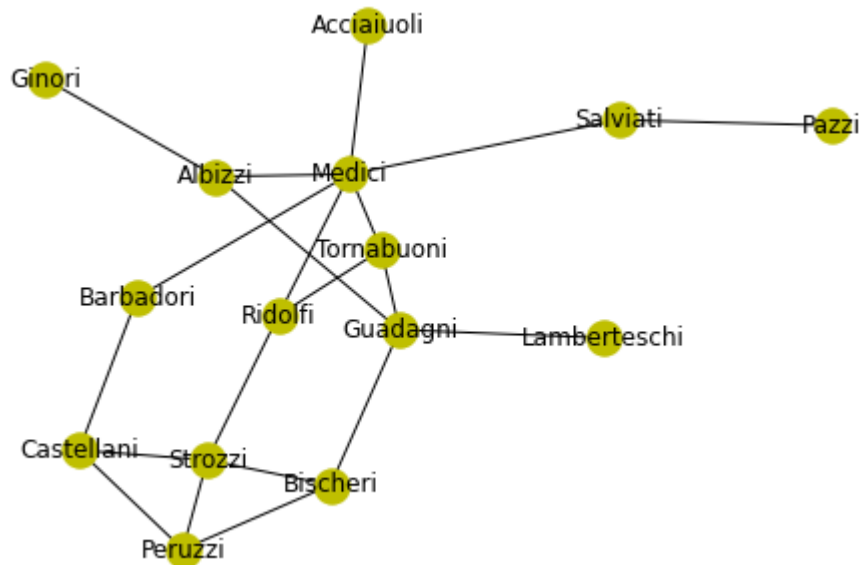
```
In [24]: nx.betweenness centrality(G)
```

```
Out[24]: {'Acciaiuoli': 0.0,
'Albizzi': 0.21245421245421245,
'Barbadori': 0.09340659340659341,
'Bischeri': 0.1043956043956044,
'Castellani': 0.05494505494505495,
'Ginori': 0.0,
'Guadagni': 0.25457875457875456,
'Lamberteschi': 0.0,
'Medici': 0.521978021978022,
'Pazzi': 0.0,
'Peruzzi': 0.02197802197802198,
'Ridolfi': 0.11355311355311355,
'Salviati': 0.14285714285714288,
'Strozzi': 0.10256410256410257,
'Tornabuoni': 0.09157509157509158}
```

```
In [25]: nx.betweenness centrality(G) == betweenness
```

```
Out[25]: False
```

```
In [26]: nx.draw(G, **opts)
```



- Finally, let's add the normalized betweenness centralities as attributes to the nodes of the graph, and display the resulting table.

```
In [27]: nx.set_node_attributes(G, betweenness, '$C_i^B$')
```

```
In [28]: pd.DataFrame.from_dict(
          dict(G.nodes(data=True)),
          orient='index'
        ).sort_values('degree', ascending=False)
```

```
Out[28]:
```

	C_i^D	c_i^E	degree	priorates	wealth	C_i^C	C_i^B
Medici	0.400000	0.430315	6	53	103	0.560000	0.521978
Guadagni	0.266667	0.289117	4	21	8	0.466667	0.254579
Strozzi	0.266667	0.355973	4	74	146	0.437500	0.102564
Albizzi	0.200000	0.243961	3	65	36	0.482759	0.212454
Bischeri	0.200000	0.282794	3	12	44	0.400000	0.104396
Castellani	0.200000	0.259020	3	22	20	0.388889	0.054945
Peruzzi	0.200000	0.275722	3	42	49	0.368421	0.021978
Ridolfi	0.200000	0.341554	3	38	27	0.500000	0.113553
Tornabuoni	0.200000	0.325847	3	n/a	48	0.482759	0.091575
Barbadori	0.133333	0.211706	2	n/a	55	0.437500	0.093407
Salviati	0.133333	0.145921	2	35	10	0.388889	0.142857
Acciaiuoli	0.066667	0.132157	1	53	10	0.368421	0.000000
Ginori	0.066667	0.074925	1	n/a	32	0.333333	0.000000
Lamberteschi	0.066667	0.088793	1	0	42	0.325581	0.000000
Pazzi	0.066667	0.044815	1	n/a	48	0.285714	0.000000

Summary

There are many different ways to be important. As a node in a network, you are important if

- you have **many friends** (degree centrality)
- you have **important friends** (eigenvalue centrality)
- you are **close** to many (closeness centrality)
- many interactions **pass through** you (betweenness centrality).

Code Corner

networkx

- `read_yaml` : [\[doc\]](#)

(https://networkx.github.io/documentation/stable/reference/readwrite/generated/networkx.readwrite.json_graph.read_graph6.py)

- `closeness centrality`: ([doc](https://networkx.github.io/documentation/stable/reference/algorithms/generated/networkx.algorithms.centrality.closeness_centrality.html))
(https://networkx.github.io/documentation/stable/reference/algorithms/generated/networkx.algorithms.centrality.closeness_centrality.html)
- `betweenness centrality`: ([doc](https://networkx.github.io/documentation/stable/reference/algorithms/generated/networkx.algorithms.centrality.betweenness_centrality.html))
(https://networkx.github.io/documentation/stable/reference/algorithms/generated/networkx.algorithms.centrality.betweenness_centrality.html)



Exercises

1. Recall that C_i^C is the normalized closeness centrality of node i . Why is $0 \leq C_i^C \leq 1$?
When is $C_i^C = 1$? Is C_i^C ever 0?
2. Recall that C_i^B is the normalized betweenness centrality of node i . Why is $0 \leq C_i^B \leq 1$?
When is $C_i^B = 1$? Is C_i^B ever 0?
3. Determine the closeness centrality and the betweenness centrality of the nodes in some random trees. What do you observe?
4. Compute the closeness centrality and the betweenness centrality of the nodes of the Petersen graph. What do you observe?

In []: